Seminário de projeto · PUC Campinas

Cylab local

Assistente open-source de segurança de código, 100% local.
Primeira entrega: arquitetura híbrida e dataset ouro v0.4-gold, com 576 pares.

O problema: três lacunas

SAST

Aponta o problema, mas a mensagem é curta, em inglês e não traz a correção.

Nuvem

Assistentes LLM na nuvem exigem enviar código-fonte, às vezes proprietário, a terceiros.

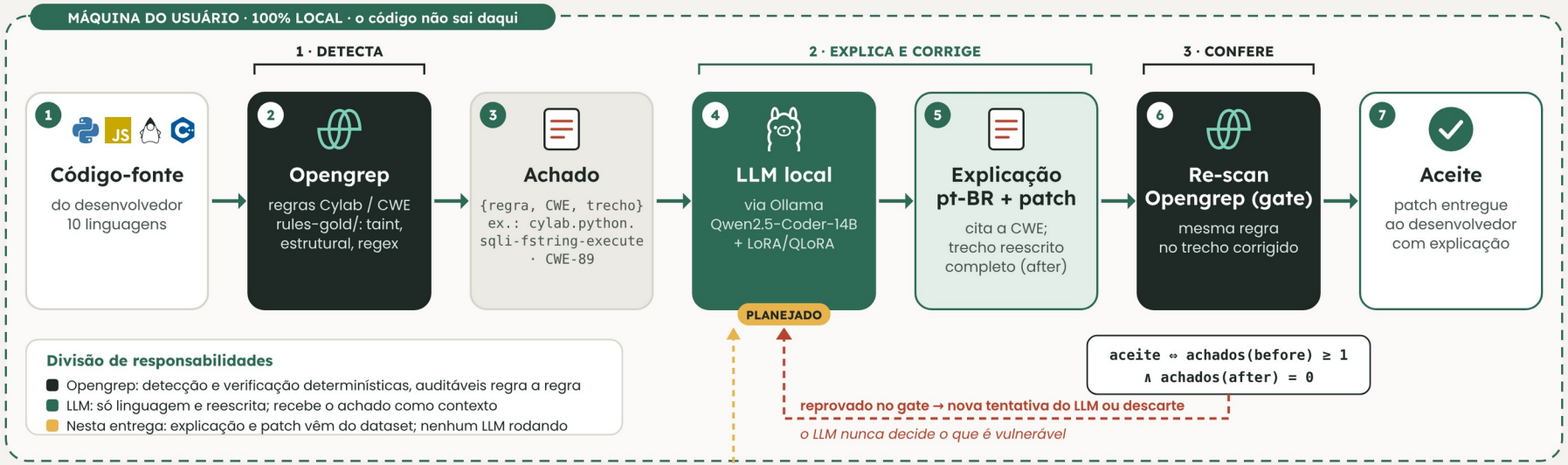
LLM só

Sozinho, o modelo alucina: inventa vulnerabilidades e deixa passar outras. Não é auditável.

Pergunta

Dá para ter explicação e correção em português, sem que o código saia da máquina e sem confiar no LLM para decidir o que é vulnerável?

Arquitetura híbrida



STACK

Linguagens do dataset

Python · JavaScript · TypeScript · Java · PHP · C# / .NET · C · C++ · SQL · PostgreSQL · Bash

Ferramentas

Opengrep 1.32 · detecção + gate · **Ollama** · planejado · **Python** · scripts do gate · **VS Code** · CLI/extensão planejadas

github.com/ArthurAlcausa/cylab-dataset

logos: simple-icons (CC0) e Opengrep; marcas de seus titulares

Por que 100% local

Quatro razões para executar localmente

Privacidade: o código-fonte não sai da máquina.
Custo: nenhum custo por consulta.
Reprodutibilidade: mesmo modelo, mesmo resultado auditável.
Acesso: uso em sala de aula e sem internet.

Plano, ainda não implementado: modelo servido via Ollama (execução local de LLMs, código aberto), com CLI e extensão para o VS Code; ajuste fino LoRA/QLoRA sobre Qwen2.5-Coder-14B.

Anatomia do par

Como ler o id: cylab-<prefixo>-<lang>-<cwe>-<nº>. “bill” é o prefixo do lote/contribuidor que escreveu o par, não um produto.

Regra: sqli-fstring-execute é uma regra de taint (valor do chamador até execute) e cobre f-string e .format(); este exemplo usa .format().

Um registro JSONL com 10 campos

id · lang · cwe · source · license
opengrep_rule · before · after
explanation_pt · split

after é o trecho corrigido completo, não um diff. Licença CC-BY-4.0; pares escritos do zero para a Cylab.

Holdout v0.4-gold · regra cylab.python.sqli-fstring-execute

cylab-bill-python-89-0002

CWE-89 | *python*

explanation_pt: “A query monta SQL com interpolação de string usando o valor de status vindo do chamador, o que permite injeção de SQL (CWE-89). A correção passa o valor como parâmetro vinculado, e o driver o trata sempre como dado.”

Como montamos o dataset ouro

1 · Escrita + schema

Pares escritos do zero para a Cylab; o validador de esquema confere os 10 campos de cada registro.

Antes do dedup, o código é normalizado: sem comentários; literais, números e identificadores viram marcadores.

Critério de partição

Train/holdout só depois do dedup: 489 treino, 87 holdout no v0.4-gold.

2 · Dedup por estrato

Jaccard (Broder) em 4-gramas de tokens, limiar 0,85, só dentro de linguagem + CWE. Sem MinHash.

Cada etapa deixa relatório e SHA-256 em data/, para auditoria.

3 · Gate rules-gold

A regra dá ≥ 1 achado no before e = 0 no after. v0.4-gold: 576/576.

O gate: critério de aceite automático

576/576

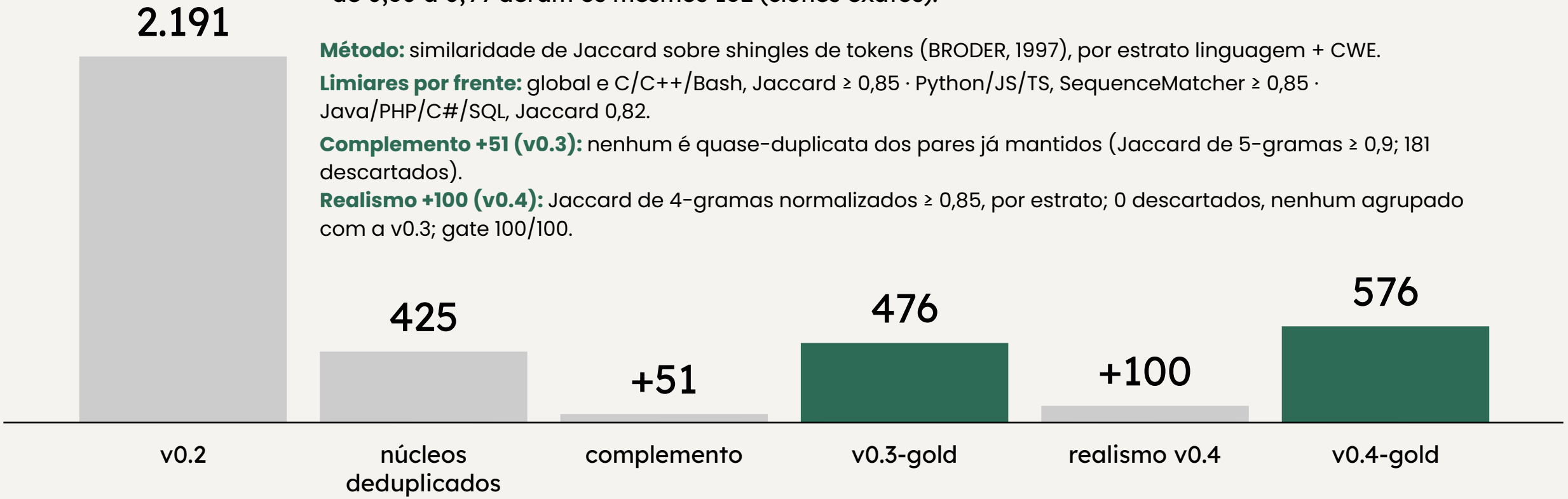
pares do v0.4-gold aprovados no gate rules-gold; o v0.3-gold (476, SHA f37e1316…) segue intacto

Aceite(p) se e somente se achados_r(before) ≥ 1 e achados_r(after) = 0. Implementado em scripts/gate_gold.py, com as regras de rules-gold/, e complementado pelo validador de esquema.
O gate garante que a falha detectada pela regra desaparece; não garante a correção funcional do patch.



De 2.191 pares ao ouro

Near-clones: variações do mesmo template. Na fatia C/C++/Bash, 602 pares viraram 182; os cortes de 0,80 a 0,99 deram os mesmos 182 (clones exatos).



Método: similaridade de Jaccard sobre shingles de tokens (BRODER, 1997), por estrato linguagem + CWE.

Limiares por frente: global e C/C++/Bash, Jaccard ≥ 0,85 · Python/JS/TS, SequenceMatcher ≥ 0,85 · Java/PHP/C#/SQL, Jaccard 0,82.

Complemento +51 (v0.3): nenhum é quase-duplicata dos pares já mantidos (Jaccard de 5-gramas ≥ 0,9; 181 descartados).

Realismo +100 (v0.4): Jaccard de 4-gramas normalizados ≥ 0,85, por estrato; 0 descartados, nenhum agrupado com a v0.3; gate 100/100.

Ouro v0.4-gold

576

pares · gate 576/576

Por linguagem: C++ 86 · Python 74 · JavaScript 68 · C 67 · Bash 67 · SQL 65 · Java 40 · PHP 39 · TypeScript 37 · C# 33

Arquivo: data/cylab-commercial-v0.4-gold.jsonl (SHA-256 ea328aef…), público em github.com/ArthurAlcausa/cylab-dataset. "commercial" vem da linha de escala (meta de ~2 mil pares na v0.2) e não restringe o uso: licença CC-BY-4.0, aberta.

10 linguagens

Todas as linguagens do escopo estão presentes; pares escritos do zero, licença CC-BY-4.0.

Partição 489 / 87

489 pares de treino e 87 de holdout. A deduplicação foi feita antes da partição.

Procedência

Escritos do zero, por frente:
Python/JS/TS 151 · Java/PHP/C#/SQL 92
C/C++/Bash 182 · complemento +51
realismo v0.4 +100

Opengrep 1.32.0 · demo_pair.sh

Números grandes: achados da regra no before → after, medidos ao vivo pelo Opengrep.

Demo: um par C e um par Python

2 → 0

C · CWE-120 · holdout
cylab-bill-c-120-0613
strcpy em buffer de 32 bytes → snprintf limitado ao destino.

1 → 0

Python · CWE-89 · holdout
cylab-bill-python-89-0002
SQL com .format() → parâmetro vinculado.
Regra taint: cobre f-string e .format().

Esta demo mostra o dataset e o gate.

Nenhum LLM está rodando: explicação e patch vêm do dataset; o Opengrep só conta os achados no before e no after.

Limitações técnicas

Análise de fluxo (taint) por linguagem

C/C++: o taint do Opengrep 1.32 não produziu achados nos testes; as regras -gold são estruturais.

Java/PHP/C#: 32 regras em taint ativo; criptografia, segredos, cabeçalhos, XXE e libxml seguem em regex.

SQL e Bash: regex restrita (o Opengrep não oferece taint para SQL; o suporte estrutural a Bash é limitado).

Pares sintéticos, em maioria curtos: mediana de 6 linhas no before nos 576 pares. Os 100 pares de realismo da v0.4 têm de 26 a 52 linhas (mediana 35), mas são 17% do conjunto e de arquivo único.

Metadados: campo source normalizado no formato do schema (URL da CWE) nos 576 pares; validação do dataset OK.

Gate não é correção funcional

Zero achados no after mostram que a regra deixou de disparar, não que o patch preserve o comportamento.

Próximos passos

1 · Ajuste fino

Qwen2.5-Coder-14B com LoRA/QLoRA; depois, um modelo maior da família Qwen.

Também: mais pares longos, na linha do buffer de realismo, e casos com múltiplos arquivos; reavaliar o taint em C/C++ quando houver suporte.

Pergunta da próxima entrega

Um modelo local, treinado nesses pares, gera patches que passam no gate?

2 · Avaliação no holdout

87 pares do holdout v0.4-gold. Critério: o patch gerado passa no gate; revisão humana das explicações.

Requisitos de hardware: medidos após o ajuste fino. Nenhuma estimativa nesta entrega.

3 · Empacotamento local

Modelo servido via Ollama, com CLI e extensão para o VS Code.

Conclusão

Primeira entrega: arquitetura definida e dataset ouro construído e verificado. Obrigado; ficamos à disposição para perguntas.

Três mensagens

1. Detecção determinística com o Opengrep; o LLM apenas explica e corrige, e o patch é conferido.
2. Dataset ouro com critério de aceite automático: 576/576 no gate, v0.4-gold.
3. Tudo local e open-source.

Referências e fontes

OPENGREP. Opengrep, release v1.32.0. github.com/opengrep/opengrep

MITRE. CWE – Common Weakness Enumeration. cwe.mitre.org

CREATIVE COMMONS. CC BY 4.0. creativecommons.org/licenses/by/4.0

OLLAMA. Ollama. ollama.com

Ferramentas e padrões

Referências completas (ABNT) no relatório acadêmico: seminar/DOC_ACADEMICO_BANCA.md. URLs acessadas em 27 out. 2026.

Literatura

HU et al. LoRA. arXiv:2106.09685, 2021.

DETTMERS et al. QLoRA. arXiv:2305.14314, 2023.

HUI et al. Qwen2.5-Coder technical report. arXiv:2409.12186, 2024.

BRODER. On the resemblance and containment of documents. 1997.

CYLAB. Repositório público: github.com/ArthurAlcausa/cylab-dataset

CYLAB. v0.4-gold, 576 registros, CC-BY-4.0: data/cylab-commercial-v0.4-gold.jsonl (SHA-256 ea328aef…)

CYLAB. v0.3-gold, 476 registros, intacto: data/cylab-commercial-v0.3-gold.jsonl (SHA-256 f37e1316…)

CYLAB. Regras e gate: rules-gold/; scripts/gate_gold.py

Artefatos do repositório Cylab